

Zakura health Endpoint Slowloris Resource Exhaustion Vulnerability Report

Summary

The health endpoint limits how quickly new connections are accepted but does not limit how long accepted connections live or how many remain active. Slow partial requests can accumulate across rate windows, exhaust file descriptors, and make real health checks and node services unavailable.

Mitigating Factors:

- The endpoint is disabled unless `health.listen_addr` is configured.
- Loopback binding or an internal-only network sharply reduces direct attacker reachability.
- A reverse proxy that enforces header, idle, lifetime, and concurrent-connection limits can provide immediate protection.
- The reviewed issue concerns availability; it does not establish data disclosure or transaction modification.

Aggravating Factors:

- The endpoint has no authentication by design once it is reachable.
- An attacker can stay below the new-connection rate threshold while old connections remain alive.
- Accepted connections have no header, keep-alive, idle, or total-lifetime deadline and no active-task semaphore.
- After descriptor exhaustion, repeated accept errors can create a CPU and log-volume amplification loop.
- Health endpoints are commonly exposed to cluster and load-balancer networks in production.

Vulnerability Details:

Status, Product, and Version

- Record status: PUBLISHED
- Product and version: Zakura / zakurad / 1.2.0
- Weakness classification: CWE-400: Uncontrolled Resource Consumption; CWE-770: Allocation of Resources Without Limits or Throttling

Cross-Validation Conclusion

The existing fixed-window rate limit bounds arrivals, not retained work. Because a partial or idle HTTP connection can outlive the window, a low-rate attacker can steadily increase active sockets and tasks until legitimate probes fail.

Trigger Conditions

- `health.listen_addr` is enabled and reachable.
- The attacker opens incomplete or persistent HTTP connections below the arrival-rate threshold.
- No reverse proxy or operating-system policy imposes a lower lifetime or active-connection cap.
- Connections remain active across multiple rate-limit windows.

Normal and Abnormal Behavior

Normal behavior

A health client sends a complete small request, receives a response, and releases the connection within a short bounded lifetime.

Abnormal behavior

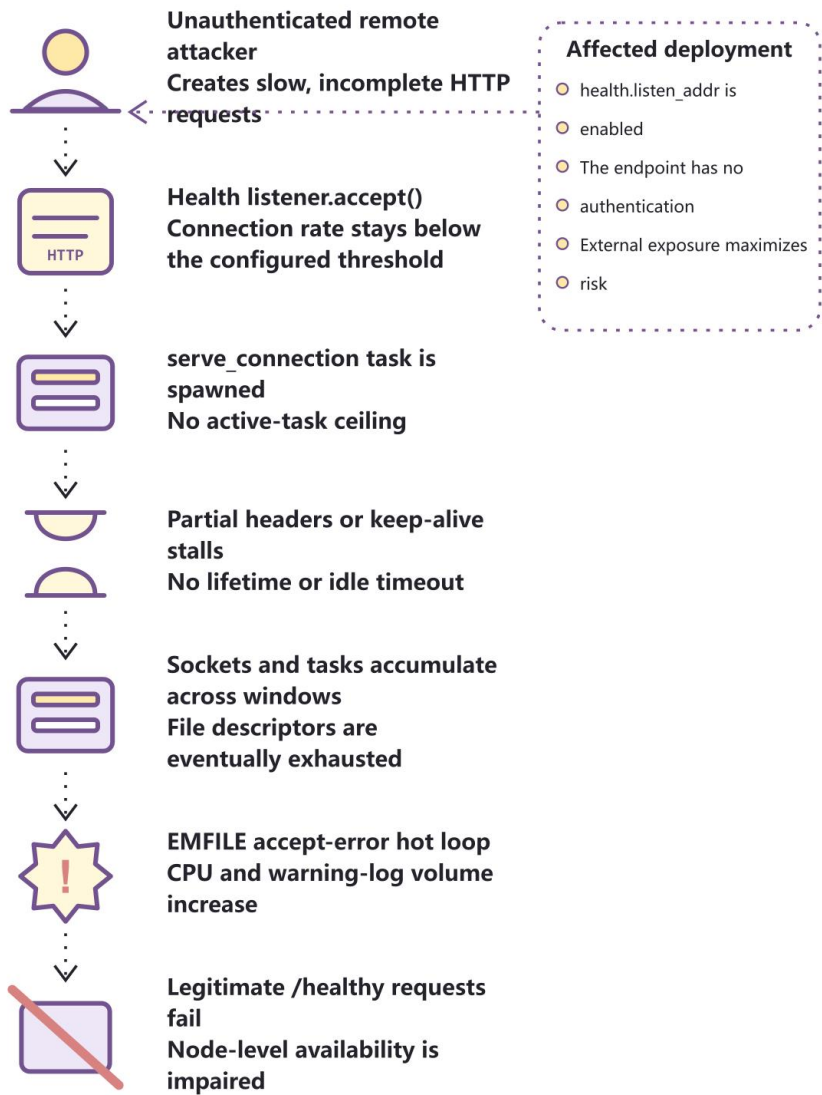
A client sends partial headers or holds keep-alive state, causing the server to retain a socket and task indefinitely while later windows admit more connections.

Full Call Chain Diagram

The complete call chain is shown below. The dashed box states the deployment condition, version boundary, or evidence boundary that controls the conclusion.

Health Endpoint Slowloris Call Chain

A new-connection rate limit does not bound long-lived partial requests



Technical Details:

Root cause 1: the limiter controls arrivals rather than concurrency

```
rate_limiter.until_ready().await;  
let (stream, _) = listener.accept().await?;  
tokio::spawn(serve_connection(stream, health_service));
```

Once accepted, a connection is no longer represented in the fixed-window admission decision.

Root cause 2: connection lifetime is unbounded

No header timeout, idle timeout, or total request lifetime forces partial requests to finish, and no semaphore limits active `serve_connection` tasks.

Root cause 3: accept errors can amplify the outage

```
loop {  
    match listener.accept().await {  
        Ok(conn) => spawn(conn),  
        Err(error) => warn!(?error, "health accept failed"),  
    }  
}
```

Minimal Reproduction Logic (Isolated Test Environment Only)

In an isolated environment, open slow incomplete HTTP requests at a rate below the configured threshold. Confirm whether active connections grow across multiple windows and whether a legitimate `/healthy` request still succeeds.

```
connect health_port  
send "GET /healthy HTTP/1.1\r\nHost: test\r\n"  
hold connection open without final CRLF
```

Security Impact

- Legitimate readiness and liveness probes fail, which can trigger unhealthy routing or restart behavior.
- File-descriptor exhaustion can affect other sockets in the same process.
- An accept-error hot loop can add CPU pressure and very large warning logs after exhaustion begins.

Long-Term Remediation

- Set short header, idle, keep-alive, and total-connection timeouts.
- Add a global active-connection semaphore and optional per-source quotas.
- Combine arrival-rate limiting with retained-work limits.
- Back off on repeated accept errors and default the endpoint to loopback or an internal management network.

Recommended Regression Tests:

- Keep a partial request open past every configured timeout and verify closure.
- Open connections below the arrival limit across multiple windows and confirm the active cap never grows.
- Verify /healthy remains responsive at the configured safe-load boundary.
- Force an accept error and verify bounded backoff without a warning-log hot loop.